Software GPU backends for TypeGPU CI

Lavapipe vs SwiftShader: what was tested, what was found, what can be claimed. Executive summary.
Mykhailo Zuiev, QA, 2026-10-20. Full evidence, scripts and tests: the study repository.

Verdict. Use **Lavapipe** (Mesa llvmpipe through apt, version-pinned) as the primary software GPU backend for TypeGPU CI. SwiftShader is **not needed for bug detection**; keep it only as an optional nightly cross-check. Confidence: medium-high for "Lavapipe rather than SwiftShader as the single backend", medium for "no second backend needed". Scope: regression detection for TypeGPU on x86_64 CI runners. Software backends do not replace tests on real GPUs.

Why

- **Detection is a tie.** 50 seeded TypeGPU bugs (18 from real fix history, 32 synthetic): Lavapipe detects 36, SwiftShader 34. The 2-bug gap is f16 only (SwiftShader has no `shader-f16`). Identical result on arm64 Docker and on native x86_64 runners. No bug is caught only by SwiftShader.
- **No flakes on either.** 62 CI jobs per backend on native ubuntu-24.04 x86_64, 6 CPU models, 3 times of day: 0 pass/fail flakes, 0 value variance (suite of 1878 tests). 87 of 97 docs examples run identically on Metal, Lavapipe and SwiftShader (the other 10 need camera, WebGL or network).
- **Lavapipe wins the tie-breakers.** f16 available; 4-6x faster compute (SwiftShader exceeds a 240 s example limit on 4 heavy examples); one apt command instead of a maintained extraction of a Chromium-bundled library; closer to Metal in the real examples (43 vs 30 of 87 identical-or-tiny).
- **SwiftShader's real advantages** are bit-stability across CPUs and low version drift (13 Chromium builds M108-M158 bit-identical except one FMA change). Both Lavapipe issues are manageable: NaN sign varies between runs (compare by category), and Mesa 26.1+ flushes subnormals (pin the version).

What GPU CI adds over TypeGPU's mocked unit tests



- Of 50 bugs: mocked unit tests catch 37 (16 of them only by WGSL snapshot, a weak signal), GPU backends catch 36; union 47. GPU-only catches (10) are mostly `std` builtins with no CPU implementation and no mocked test, plus `dispatchThreads` rounding and `@typegpu/sort`. GPU CI is complementary, not a replacement. A cheaper partial fix also exists: add tests for those builtins.

Scorecard (criteria fixed before any data)

Criterion	Lavapipe	SwiftShader	Result
C1 Detection (50 bugs)	36 detected, 9 missed	34 detected + 2 not run (f16), 9 missed	Tie (same on x86_64)
C2 Flakes (62 jobs/backend)	0	0 (v1); 1 timeout flake on a heavy example	Tie
C3 Examples (97)	87 run clean	87 run clean	Tie
C4 Truth proximity	CTS rejects 0 of 195 210 lanes	same; lean: SwiftShader off in more groups	Strict tie, leans Lavapipe
C5 f16	available	not offered by any of 16 builds	Lavapipe
C6 Update drift	11 Mesa versions: stable to 26.0, 3% lanes move at 26.1	13 builds: bit-identical M108-M158 (1 FMA step)	SwiftShader slightly
C7 Cost	apt 14 s; examples 17 min; no timeouts	.deb 13 s; 3.7-6.2x slower dispatch; timeouts	Lavapipe

What can be claimed, and what cannot

Can claim (evidence is direct)	Cannot claim
Equal detection on this bug set on both architectures, with repeat runs. No backend leaves the CTS tolerance (0 rejected lanes of 195 210). No flakes on the main suite across 124 jobs. GPU tests catch 10 bugs the mocks miss.	That either backend matches a real GPU: races (M35) were seen only on Metal, and out-of-bounds writes behave differently (M32). Long-run flake rates (measured over about 3.5 h on one day). Detection rates for bugs outside this catalogue (it is ours; 14 GPU tests were written after seeing it). Mesa 26.1+ on x86_64, or bugs about NaN and subnormals (none in the catalogue).

Recommended setup

- Install `mesa-vulkan-drivers libvulkan1` on ubuntu-24.04 (Mesa 25.2.8), assert backend identity in every job, and keep the dawn.node object referenced (a garbage-collected `create()` result crashes the process).
- PR tier: fast subset (about 3.5 min/job); nightly: all 97 examples (about 17 min, example timeout at least 300 s). Assert via CTS intervals, categories and exact integers, not golden floats or NaN bits.
- Keep real-GPU/Metal checks for what emulators cannot see. Optional: SwiftShader nightly with a sha256-pinned library.

What was tested

Area	Scale
CPU vs GPU numeric oracle	198 759 lanes per configuration (groups A-D, f16, mod/log/log2/pow); Metal, Lavapipe arm64, native x86_64 on 5 CPU models, SwiftShader arm64 and x86_64; real WebGPU CTS intervals
WebGPU CTS	1159 cases Metal, 1162 Lavapipe (arm64 and x86_64), 686 + 476 skipped (f16) SwiftShader
Test suites	426-test ported std suite, six investigation suites (1433 tests), 13 TypeGPU example tests
TypeGPU docs examples	all 97 run headless on Metal, Lavapipe, SwiftShader, two repeats each
Seeded bugs	50 (18 historic, 32 synthetic) on 3 backends on arm64, replicated on native x86_64 (2 backends), and against TypeGPU's 3576 mocked unit tests
CI soak	299 jobs on native ubuntu-24.04 x86_64, 6 CPU models; suite, bit-level dump, examples, performance
Version drift	11 Mesa versions (23.2 to 26.2) and 13 SwiftShader builds (Chromium M100 to M158), plus 2 source builds

Detailed report

Every section ends with the repository paths that hold the code, data and tables behind it. Raw run directories (several GB) are not in the repository; the documents identify them by manifest hashes.

1. Question, scope and method

TypeGPU generates WGSL that runs on the GPU. CI machines normally have no GPU, so the generated shader code is not exercised by the mocked unit tests. The study asks which software Vulkan implementation (Lavapipe, SwiftShader) should back real WebGPU tests in CI, and what such tests add. Both run through the same path: Node 24, `webgpu@0.6.2` (dawn.node, Dawn commit 77f9a9aa), typegpu 0.12.7, vitest, in Docker or on native runners.

Decision criteria C1-C7 and the decision rule were written before any Phase-3 data and were not changed afterwards. Terms: **accepted** = value inside a bounded interval produced by the real WebGPU CTS implementation (not proof of correctness); **rejected** = outside one (not by itself a defect); **indeterminate** = NaN operand or unbounded interval; **unverified** = nothing to judge. Green is not correct and red is not a defect.

Files: docs/PLAN.md (pre-registered criteria), docs/FINAL_DECISION.md (verdict, scorecard, limits), docs/INDEX.md (claim to folder map).

2. Phases and environments

Phase	What	Key facts
1 Lavapipe baseline	Frozen oracle of CPU vs GPU results; Metal as reference run	Mesa 25.2.8 / LLVM 20.1.2. 198 759 lanes per configuration over groups A-D, E (f16), L (mod, log, log2, pow). rejected = 0 everywhere. Evaluator self-test 49/49. Real CTS: 1159 cases pass on Metal, 1162 on Lavapipe (arm64 and x86_64).
2 SwiftShader	Same path with SwiftShader from Debian chromium-common 155.0.8059.79 (arm64 LLVM JIT, amd64 Subzero JIT), sha256-pinned	No shader-f16. CTS 686 pass, 476 skipped (f16), 0 fail. Full suite 426 tests: Lavapipe 394/15/17, SwiftShader arm64 371/22/33, x64 359/34/33 (pass/fail/skip). Crash root cause found: dawn.node tears down if the create() result is garbage-collected.
3 Decision study	Eight workstreams: seeded-bug detection, mocked-test baseline, 97 examples, divergence adjudication, drift (Mesa, SwiftShader), CI soak, native x86_64 replication	299 CI jobs, 11 Mesa and 13 SwiftShader builds, 50 seeded bugs on 3 backends (arm64) plus a native x86_64 replication.

Backend identity is asserted on every run (Vulkan vendor/device, ICD hash, no foreign ICD, adapter fields, a negative control). Pins: Node 24.21.0, webgpu 0.6.2, typegpu 0.12.7, vitest 4.1.11, CTS commit e325d0bcc2cb, Ubuntu 24.04 image digest plus snapshot date for the pinned Lavapipe image.

Files: docs/phase1/BASELINE.md, docs/phase2/COMPARISON_REPORT.md, docs/phase2/FINAL_REPORT.md, docker/, harness.ts, evaluator/, lib/.

3. C1: bug-detection power

Method. A catalogue of 50 mutants was built from the git history of TypeGPU (18 real fixes reversed, hashes and subjects recorded) and 32 synthetic bugs of classic classes (struct offsets, vec3 padding, matrix layout, integer wrap, bit ops, argument order of std functions, barriers, defaults). Each patch applies cleanly to a pristine copy of HEAD e893ad11 and changes only `packages/*/src`; 16 were additionally executed on the CPU path. The suites run against the mutated source (not the npm build; canaries M24 and M26 prove it). **Detection** = pass to fail, or a value fingerprint change in a test whose fingerprint was identical across two baseline repeats, or a crash. A change visible only in generated WGSL text is reported separately.

Results. Per-mutant verdicts are identical on 48 of 50 mutants; they differ only on M01 and M21 (f16-only). The nine misses are the same on both backends. Restricted to the 41 mutants with a named reach, both detect 32. The 14 "own" tests were written after reading the catalogue (10 mutants had no reaching test), so the pre-existing pool is also reported: 26 detected on each emulator.

Pool	Lavapipe	SwiftShader	Metal
Pre-existing GPU pool	26 detected / 5 text-only / 19 missed	25 + 1 not run (f16) / 5 / 19	26 / 5 / 19
Plus 14 own tests	36 / 5 / 9	34 + 2 not run (f16) / 5 / 9	36 / 5 / 9
Native x86_64 replication (8 CI jobs, 5 CPU models)	36 / 5 / 9 (identical)	34 + 2 / 5 / 9 (identical)	n/a

Differences between backends and Metal. M35 (a removed workgroup barrier, a race) was detected only on Metal; the emulators execute in lock-step and show a text change only. M32 (guard `>` instead of `>=`) was detected on both emulators but not on Metal, because the emulators clamp an out-of-bounds storage write to the last element while Metal drops it (verified by a probe on all three).

x86_64 replication. Pre-registered hypothesis: verdicts on native x86_64 equal arm64 and the two backends differ by fewer than 2 mutants (excluding f16). Confirmed: 100 of 100 verdict classes identical. The only effect of the Subzero JIT: 14 tests change detection channel for M33 (12 min/max/neg tests are already red in the x86_64 baseline). A same-runner baseline-then-mutants differential cancels CPU differences. About 159 billable CI minutes.

Files: mutation/CATALOGUE.md, mutation/catalogue.json, mutation/patches/, results/mutation/RESULTS.md, results/mutation/ATTRIBUTION.md, mutation/harness/, mutation/own/, results/x64-mutation/RESULTS.md.

4. What mocked unit tests already catch

TypeGPU's unit tests (3576 tests, mocked WebGPU, WGSL inline snapshots, docs example snapshots) were run on all 50 mutants after proving a healthy baseline (3574 pass, 2 skipped, two runs). Result: 21 caught by an assertion, 16 only by a WGSL snapshot (weaker: a snapshot cannot tell a bug from an intended change), 13 missed. Joined with the GPU result: **26 both, 10 GPU only, 11 mocks only, 3 neither (union 47)**.



- **GPU only (10):** M26 countLeadingZeros emits countTrailingZeros, M27 extractBits arguments swapped, M33 dispatchThreads uses floor instead of ceil, M34 prefix-scan barrier removed, M42 bitonic default padding, M46 insertBits, M47 firstLeadingBit, M48 countTrailingZeros, M49 dot4U8Packed, M50 trunc emits floor. Seven are std builtins without a CPU implementation; `@typegpu/sort` has no unit tests.
- **Mocks only (11):** host-side logic, CPU-only paths and layout-writer tests (M04 M05 M06 M08 M11 M12 M13 M15 M16 M20 M44). **Neither (3):** M02, M25 (faceForward, text-only on GPU), M35 (needs Metal).

Files: results/mock/RESULTS.md, results/join-mutation-vs-mock.md (per-mutant table, also in the appendix).

5. C2 and C7: CI soak on native x86_64

A workflow without Docker emulates what a team would add: install the backend, run the suite, emit per-test status and value fingerprints. 299 matrix jobs, 0 job-level infrastructure failures, identity asserted in all.

Measure	Lavapipe	SwiftShader
Suite v1 (426 full + 1436 investigation + 16 example tests), 62 jobs/backend, 6 CPU models, 3 times of day	0 flakes, 0 value variance	0 flakes, 0 value variance
Outcome totals (full suite, identical in every job)	394 pass / 15 fail / 17 skip	359 / 34 / 33
Bit-level dump, 60 jobs/backend, 195 210 lanes	186 lanes vary, all NaN sign/payload only (7fc00000 vs ffc00000), random per run, never a finite value	1 lane (inverseSqrt(-0.5)) flips sign in 3/60 jobs on AVX-512 EPYC
Examples (97, one repeat)	classes identical across jobs	same, except ripple-cube: 4 ran / 4 timed out (limit about 455-476 s)
Job wall time, v1 (median / p95)	204 s / 235 s	152 s / 169 s
Backend acquisition	apt 13.6 s (17.5 MB)	pinned .deb 12.8 s (31 MB, sha256-checked, about 250 downloads OK)
All 97 examples per job	17 min; slowest ripple-cube 196 s (91% of 240 s limit)	13 min + 34 min for 8 heavy ones; ripple-cube 474 s, attractors 380 s
Steady dispatch ratio, 15 jobs x 6 pairs (shared noisy runner)	1x	3.7-6.2x slower (compile 3.5-4.5x, readback faster)

Upper bound: zero flakes in 62 jobs gives a 95% upper bound of about 4.7% per job. The window is about 3.5 hours on one day, two runner images and one Mesa version; long-run flake rate is unknown. About 1375 billable CI minutes were used (estimate).

Files: results/soak/RESULTS.md, .github/workflows/ci-soak.yml, ci/.

6. C3 and C5: all 97 docs examples

A generic runner executes each example in its own process on a headless dawn.node (fake DOM, offscreen canvas, pinned time and randomness, 3 and 30 frames, two repeats), records errors, validation messages, device features and readbacks, and compares backends.

- **Classes.** 87 ran clean and 10 need an unavailable API on every backend (5 camera/video, 3 WebGL, probability via WebGL charting, compute-cloth offline fetch). 0 validation errors, 0 throws, 0 hangs; 0 false reds against Metal. 85/97 produce a checkable non-zero readback, and 7 ran with degraded inputs (fake 2D canvas or audio), so real-input coverage is roughly 78-85 of 97. "Runs clean" is a coverage statement, not correctness.
- **f16.** Only 3 examples touch shader-f16, all as optional (mnist-inference, monocular-light-injection, log-test); SwiftShader takes the f32 path; none fail. With f16 hidden the generated WGSL is identical on all backends.

- **Cross-backend at checkpoint 3** (bit-identical / tiny / small / moderate / large): Metal-Lavapipe 16/27/15/16/6, Metal-SwiftShader 13/17/27/17/8, Lavapipe-SwiftShader 15/14/25/17/11. Lavapipe is closer to Metal (43 vs 30 identical-or-tiny). SwiftShader is the lone outlier in 16 examples, Lavapipe in 1 (bitonic-sort, whose array size depends on adapter limits).
- **Root causes found.** F32_MAX written into an r16float texture: Metal and Lavapipe store 65504, SwiftShader NaN (spec-indeterminate, non-portable example). -(+0) is -0 on Metal and Lavapipe, +0 on SwiftShader (allowed by WGSL). The game-of-life difference seen earlier is by design: naive treats out-of-range neighbours as dead, tiled and bitpacked wrap; each matches a CPU model bit-exactly on all backends.
- **Heavy examples:** SwiftShader needs an example timeout of at least 450 s; Lavapipe is slow on ripple-cube (about 183-196 s).

Files: results/examples/RESULTS.md, examples-runner/, typegpu-examples/.

7. C4: who is the outlier when the backends differ

Every place where Lavapipe and SwiftShader disagree was grouped into 16 root causes and scored with the strongest oracle available: spec rule, then the real CTS interval, then Metal as a non-authoritative reference.

Outcome	Groups	Weighted by TypeGPU relevance
Strict: Lavapipe outlier / SwiftShader outlier	0 / 0	0 / 0
Both fine / indeterminate / f16 capability gap	10 / 5 / 1	14 / 4 / 3
Lean (IEEE + Metal tie-break, non-authoritative): SwiftShader off / Lavapipe off / mixed	6 / 3 / 6	9 / 3 / 6

- The CTS cannot name an outlier: rejected = 0 for Lavapipe, both SwiftShader builds and Metal over 195 210 lanes. The tie-break leans to Lavapipe through sin/cos and the f16 gap.
- SwiftShader sin/cos error is about 1.9e-4 (Lavapipe 5e-8, Metal 1e-7): inside the CTS interval (at most 39% of the half-width) but visible in the probability example (unit circle check). SwiftShader also differs on special inputs where the spec is silent: log(0) = -88, exp(NaN) = +inf (Lavapipe matches Metal and IEEE on all 80 lanes).
- Two Phase-2 statements were softened: the claim that Lavapipe asin/acos exceed the bound holds only for the harness's abs-only bound (the real CTS interval accepts every sampled lane); the 12 SwiftShader x86_64 min/max/neg reds are harness-policy artefacts (the CTS accepts flushed values).
- Of 53 divergent examples: backend numerics (X1, X4, X5, X6, X8), example nondeterminism or environment (X3, X7), capability (X2); no correctness difference against the spec was established for any backend.

Files: results/adjudication/SCOREBOARD.md.

8. C6: what changes when the version changes

Lavapipe (Mesa), arm64 Docker, all workloads including CTS. Mesa 22.0.1 and 22.3.6 install but dawn.node gets no adapter, so 23.2 is the oldest usable. Lanes counted out of 198 759.

Mesa version	Lanes changed	Full suite pass/fail/skip	Investigation failures (of 1433)	Other
23.2.1, 24.0.5, 25.0.7, 25.2.8 (baseline)	0	394/15/17	14	only the probability example moves at 25.0 to 25.2 (normalize, 2 ULP)
26.0.8	0	394/15/17	14	none
26.1.6	5 980 (3.0%)	402/7/17	12	subnormal operands flushed to zero
26.2.4, 26.2.5	5 980	402/7/17	4	asin/acos accuracy fixed; CTS 1162/1162 on every version

Category changes are 0 in every step, all test flips go fail to pass, and the cause of the 26.1 step is Mesa, not LLVM (same on LLVM 19, 21, 22). Repeat noise is zero. arm64 vs x86_64 on the same Mesa differs only in NaN payload. Pinning is necessary for exact-value tests and sufficient for run-to-run reproducibility on one architecture.

SwiftShader, arm64 Docker. 13 distinct vendored builds were run (Electron M100, M108, M120; Debian M128-M130, M138, M146, M150, M154; Chrome for Testing 155, 157 beta, 158 canary), plus source builds of the Dawn pin and of Chromium's M155 revision.

- Builds M108 to M158 are bit-identical to M155 on all 195 210 lanes; M100 differs on 420 lanes (0.2%). Categories, CTS (686/476/0), investigation suites and examples show no change; shader-f16 is never offered.
- The probability example differs for M100 and M108-M141 and not from M142: fused multiply-add is on in M108-M141 and off from M142 (689 of 8011 probe inputs, 1 ULP). M100 to M108 rewrote sin/cos/tan/log/log2/pow/normalize/dot.

- A source build of the same revision differs from every vendored build on 2.4% of lanes (exp, log, pow families); the Dawn pin is not interchangeable with the Chromium-bundled library. The Subzero JIT cannot be built for arm64.
- Two of 27 full-suite runs on arm64 Docker had a vitest worker crash (M128, M150, not reproducible); none occurred in 62 native SwiftShader jobs. Lavapipe had none in 11-14 runs, which does not statistically separate the backends (Fisher p between 0.54 and 1).

Files: results/drift-lavapipe/RESULTS.md, results/drift-swiftshader/RESULTS.md, docker/drift-lavapipe/, docker/drift-swiftshader/.

9. Findings independent of the choice of backend

- TypeGPU docs example `rendering/disco`: the dev-only "Test Resolution" control emits WGSL calling an undeclared function (5 compile errors) on all backends.
- dawn.node 0.6.2 does not understand `Float16Array`: `queue.writeTexture` sees 0 bytes and `queue.writeBuffer` silently writes zeros (affects rendering/area-light).
- dawn.node tears down the process if the `create()` result is garbage-collected; keep it referenced.
- HEAD examples log deprecation warnings (`>>=` on u32 in prefix-scan, implicit i32 to f32 conversions).
- From Phase 1: 280 std.mod mismatches came from an over-strict comparison policy, not from a backend defect; log/log2 on subnormal inputs and strict CPU/GPU equality tests remain open decisions for the team.

10. Decision rule applied and recommended recipe

Rule (pre-registered): PRIMARY must pass the hard gates (no unattributed flakes, at least 80% of examples checkable, f16 not a blocker for the wanted suite). Among passers, prefer higher C1 (a difference counts only if it covers at least 2 mutants and is not an f16 or feature gap), then C6, then C7. SECONDARY only if it adds an exclusive catch or exclusive truth check.

Both backends pass the gates. C1 ties, so C6 (SwiftShader slightly better) and C7 (Lavapipe clearly better) decide, with C5 and the examples evidence favouring Lavapipe: **Lavapipe primary**. SwiftShader adds no exclusive catch, so a second backend is not required.

Item	Recommendation
Backend	apt `mesa-vulkan-drivers libvulkan1` on ubuntu-24.04 (Mesa 25.2.8, identical in all 299 jobs); runner image bump to ubuntu-26.04 gives Mesa 26.0.8, with 0 lanes changed in our data
Pinning	Pin by version only while the mirror carries it; assert Mesa version and backend identity in every job; compare by CTS category or exact integers; canonicalise NaN
Tiers	PR: fast subset (about 3.5 min); nightly: all 97 examples (about 17 min, timeout at least 300 s)
Add tests	Eight std builtins without a CPU implementation (faceForward, countLeadingZeros, extractBits, insertBits, firstLeadingBit, countTrailingZeros, dot4U8Packed, trunc) and @typegpu/sort have no mocked tests
Keep real GPUs	Races (M35), out-of-bounds write semantics (M32) and vendor float behaviour need Metal or a hardware runner
Optional	SwiftShader nightly cross-check with a sha256-pinned library, expecting no f16 and example timeouts of at least 450 s

11. Limitations and review notes

- The mutation catalogue is ours: detection rates describe this bug set. Some mutants (9 to 13, depending on the tagging pass) were deliberately given no reaching test; 14 GPU tests were written after seeing the catalogue (the pre-existing pool alone gives 26 vs 25+1, still a tie). Baseline determinism rests on two repeats per backend.
- Drift, examples and most mutation runs are arm64 Docker; native x86_64 covers the soak (suite, bit dump, examples classes, performance) and the mutation replication. Mesa 26.1+ on x86_64 and detection depth outside the pools that detected a bug on arm64 are untested.
- Full 97-example runs of the mutants were done on Metal only (16 mutants, 3 extra detections). Example divergences are mostly unattributed numerics or example nondeterminism; "SwiftShader is the outlier" is a lean, not a verdict.
- Review addendum: pairing makes the "13 unreachable" mutants harmless for the comparison; the "eight builtins" wording was corrected (seven among the ten GPU-only catches); the two worker crashes and the 0-infra-failure statement come from different samples (arm64 Docker vs native CI); "ran clean" is not correctness; the arm64-to-x86_64 detection gap was closed by the replication.
- Integrity: frozen Phase-1 manifest 825/825 and Phase-2 manifest 4634 OK. TypeGPU source used throughout: commit e893ad11.

12. Reproduce and navigate

What	Where
Start here, prerequisites, copy-paste commands	README.md
Verdict, scorecard, limits, review addendum	docs/FINAL_DECISION.md
Evaluator, harness, CTS oracle	harness.ts, evaluator/, lib/, selftest/
Suites (ported, investigation, 13 examples)	ported/, investigations/, typegpu-examples/
All-examples runner	examples-runner/
Mutation catalogue, patches, harness, own tests	mutation/ (catalogue.json, patches/, own/, harness/)
Docker images and run scripts	docker/
CI workflows (soak, x86_64 mutation, SwiftShader, compare)	ci/, .github/workflows/
Result tables and manifests (no raw data)	results/

Appendix A. Per-mutant results

Mocked = TypeGPU unit tests; GPU columns = value-channel verdict (DET detected, txt text-only change, miss, f16 not run). Source: results/join-mutation-vs-mock.md.

ID	Bug	Mocked	Lavapipe	SwiftShader	Metal
M01	f32->f16 bit conversion rounds half-up and double-rounds subnormals	assert	DET	f16	DET
M02	Perlin dynamic cache never clears its dirty flag	MISSED	miss	miss	miss
M03	tgpu.unroll over a range ending at 0 is dropped	assert	DET	DET	DET
M04	withIndexBuffer ignores the element-count size argument	assert	miss	miss	miss
M05	isCloseTo for vectors emits "(lhs - lhs) + precision" (lhs evaluated twice)	snapshot	miss	miss	miss
M06	CPU std.saturate throws for float vectors	assert	txt	txt	txt
M07	Integer-valued literals >= 2^63 typed as abstractInt (FLT_MAX in oklab)	snapshot	DET	DET	DET
M08	Compiled writer leaves mat3x3f column padding unwritten	assert	miss	miss	miss
M09	Buffer allocations/transfers no longer padded to a multiple of 4 bytes	assert	DET	DET	DET
M10	Non-finite comptime values are no longer rejected (Finite Math Assumption)	assert	DET	DET	DET
M11	Auto vertex output does not receive fragment-determined varying locations	snapshot	miss	miss	miss
M12	withLocations treats location 0 as "no location"	assert	txt	txt	txt
M13	Global identifier resolved inside a nested call can capture a caller-local name	snapshot	txt	txt	txt
M14	Implicit conversion i32/u32 <-> abstractFloat picks i32/u32 (truncates the float)	assert	DET	DET	DET
M15	hasIndexBuffer is a constant false field	assert	miss	miss	miss
M16	Piping a configuration duplicates its bindings	assert	miss	miss	miss
M17	Generated identifiers may shadow WGSL template enumerants (read, write, uniform, rgba8unorm...)	snapshot	DET	DET	DET
M18	Unary minus on u32 no longer rejected by TypeGPU	snapshot	DET	DET	DET
M19	Compiled writer uses unpadded element stride for arrays of vec3	assert	DET	DET	DET
M20	Struct size loses trailing padding	assert	miss	miss	miss
M21	vec3<f16> alignment 4 instead of 8	assert	DET	f16	DET
M22	mat3x3f columns written packed (36 bytes) by the compiled writer	assert	DET	DET	DET
M23	Compiled writer transposes matrices given as mat objects	assert	DET	DET	DET
M24	std.mix emits mix(b, a, t)	snapshot	DET	DET	DET
M25	std.faceForward emits faceForward(e1, e3, e2)	MISSED	txt	txt	txt
M26	std.countLeadingZeros emits countTrailingZeros	MISSED	DET	DET	DET
M27	std.extractBits emits (offset, count) swapped	MISSED	DET	DET	DET
M28	CPU/comptime smoothstep has no clamp	assert	DET	DET	DET
M29	CPU/comptime mod uses floored modulo	assert	DET	DET	DET
M30	i32() cast saturates instead of wrapping	assert	DET	DET	DET
M31	for-of over std.range runs one extra iteration	snapshot	DET	DET	DET
M32	Guarded compute pipeline guard uses ">" instead of ">="	snapshot	DET	DET	txt
M33	dispatchThreads computes workgroup count with floor instead of ceil	MISSED	DET	DET	DET
M34	Workgroup barrier removed from the prefix-scan up-sweep	MISSED	DET	DET	DET
M35	Workgroup barrier removed between down-sweep and the final read	MISSED	txt	txt	DET
M36	std.atomicSub emits atomicAdd	snapshot	DET	DET	DET
M37	rg* texture formats get the wrong bytes-per-texel (default 4)	assert	DET	DET	DET
M38	u32To01F32 drops the lowest mantissa bit	snapshot	DET	DET	DET
M39	GPU-side hash multiplier differs from the CPU one by one bit	snapshot	DET	DET	DET
M40	srgbToLinear uses gamma 2.2 instead of 2.4	snapshot	DET	DET	DET
M41	opSmoothUnion blend term loses a factor h	snapshot	DET	DET	DET

ID	Bug	Mocked	Lavapipe	SwiftShader	Metal
M42	Bitonic sort default paddingValue 0x7fffffff instead of 0xffffffff	MISSED	DET	DET	DET
M43	std.select emits select(t, f, cond)	snapshot	DET	DET	DET
M44	CPU .rgb swizzle returns (r, b, g)	assert	miss	miss	miss
M45	rotationY4 WGSL sign error	snapshot	DET	DET	DET
M46	std.insertBits emits the base value and the inserted bits swapped	MISSED	DET	DET	DET
M47	std.firstLeadingBit emits firstTrailingBit	MISSED	DET	DET	DET
M48	std.countTrailingZeros emits firstTrailingBit	MISSED	DET	DET	DET
M49	std.dot4U8Packed emits dot4I8Packed	MISSED	DET	DET	DET
M50	std.trunc emits floor	MISSED	DET	DET	DET

Highlighted rows: missed by the mocked tests but detected by the GPU.

Appendix B. Terms

Term	Meaning
Lavapipe / llvmpipe	Mesa software Vulkan driver (CPU, LLVM JIT)
SwiftShader	Google software Vulkan implementation bundled with Chromium; JIT is LLVM (arm64) or Subzero (x86_64 builds used here)
dawn.node	Node binding of Dawn (WebGPU), package webgpu@0.6.2
CTS	WebGPU Conformance Test Suite; its f32/f16 interval implementation is the oracle used here
Mutant / seeded bug	A small patch that reintroduces a plausible TypeGPU bug into a copy of the source
Value / text channel	Hash of GPU readbacks vs hash of generated WGSL modules
Flake	Pass/fail status of a test differing between jobs of the same backend
Bit variance	Value fingerprint differing between jobs while pass/fail is identical